

COSMIC Full Function Points (COSMIC FFP) in Software Project Management

1. What is COSMIC?

COSMIC stands for **Common Software Measurement International Consortium**.

COSMIC Full Function Points (COSMIC FFP) is a **software sizing method** used to measure the **functional size of software**.

It is especially useful for estimating:

- Software effort
- Development cost
- Development time
- Productivity
- Project size

Simple Definition

COSMIC Function Points is a standardized method for measuring the functional size of software by counting the data movements that occur across the software's functional processes.

The important idea is:

COSMIC measures what the software does in terms of data movement, rather than counting lines of code.

2. Why Do We Need COSMIC?

Suppose a client says:

"Build an online banking application."

Before development, the project manager needs to estimate:

- How large is the software?
- How much effort will be required?
- How many developers are needed?
- How much will development cost?
- How long will development take?

Counting **Lines of Code (LOC)** is difficult before coding starts.

For example, we don't know whether the application will require:

20,000 or 50,000 lines of Java/Python code.

COSMIC provides another approach.

Instead of asking:

"How many lines of code will we write?"

we ask:

"What data does the software receive, produce, read, and write?"

3. COSMIC Measures Functional Size

COSMIC measures **functional size**.

Functional size means:

The amount of functionality that software provides to its users.

For example, an ATM provides functions such as:

- Enter PIN
- Check balance
- Withdraw money
- Deposit money
- Print receipt

COSMIC looks at the **data movements involved in these functions**.

4. Basic COSMIC Concept

COSMIC divides software functionality into:

Functional Processes

A **functional process** is a set of activities that together achieve a meaningful task for a user.

Examples:

- Login
- Withdraw cash
- Create account
- Place order
- Generate invoice

Within these processes, data moves between:

- User and software
- Software and persistent storage
- Software and external systems

COSMIC measures these **data movements**.

5. Four Types of COSMIC Data Movements

This is the **most important concept** for exams.

COSMIC identifies **four types of data movements**:

1. Entry (E)

2. Exit (X)

3. Read (R)

4. Write (W)

Remember:

E → X → R → W

Let's understand each.

6. Entry (E)

An **Entry** occurs when data moves **from a user or external system into the software**.

Example:

A user enters:

Username
Password

into a login application.

The movement:

User → Software

is an **Entry**.

Examples:

- Login details
- Customer registration data
- Payment request
- Search query
- ATM PIN

7. Exit (X)

An **Exit** occurs when data moves **from the software to a user or external system**.

Example:

The software sends:

Login Successful

to the user.

Software → User

This is an **Exit**.

Examples:

- Bank statement
- Search results
- Invoice
- Error message
- Payment confirmation

8. Read (R)

A **Read** occurs when the software retrieves data from **persistent storage**.

Database → Software

Example:

During login:

Software → Database

asks for the stored password information.

Database → Software

returns the user record.

The second movement is a **Read**.

Examples:

- Reading customer details.
 - Reading product information.
 - Reading account balance.
 - Reading employee records.
-

9. Write (W)

A **Write** occurs when software stores or updates data in persistent storage.

Software → Database

Example:

When a customer creates an account:

Software → Customer Database

The customer information is stored.

This is a **Write**.

Examples:

- Creating a new customer record.
 - Updating account balance.
 - Saving an order.
 - Storing payment information.
-

10. The COSMIC Measurement Unit

Each data movement is counted as:

1 COSMIC Function Point (CFP)

Therefore:

Data Movement COSMIC CFP

Entry (E)	1 CFP
Exit (X)	1 CFP
Read (R)	1 CFP
Write (W)	1 CFP

Formula

COSMIC Functional Size = $E + X + R + W$

This is the most important formula for basic COSMIC questions.

11. Simple Example: Login System

Suppose a login process works like this:

```
User
↓
Username + Password
↓
Software
↓
Read User Database
↓
Verify Credentials
↓
Software
↓
Login Result
↓
User
```

Now identify the movements.

Step 1 — Entry

Username and password:

User → Software

= **1 Entry**

Step 2 — Read

Software retrieves user information:

Database → Software

= **1 Read**

Step 3 — Exit

Login result:

Software → User

= **1 Exit**

Therefore:

CFP=E+R+X CFP=1+1+1 3 CFP

So this functional process has a size of **3 COSMIC Function Points**.

12. Example: Online Shopping — Place Order

Suppose a customer places an order.

Data movements:

1. Customer sends order information → **Entry**
2. System reads product information → **Read**
3. System reads customer information → **Read**
4. System writes the new order → **Write**
5. System sends order confirmation → **Exit**

Therefore:

Movement Count

Entry	1
Read	2
Write	1
Exit	1

Total:

$$1+2+1+1=5$$

Functional Size:

5 CFP

13. COSMIC Software Model

COSMIC considers three major aspects:

1. Functional Users

Entities interacting with the software.

Examples:

- Customer
- Administrator
- ATM
- External payment system

2. Functional Processes

Tasks performed by the software.

Examples:

- Login
- Withdraw money
- Place order
- Generate report

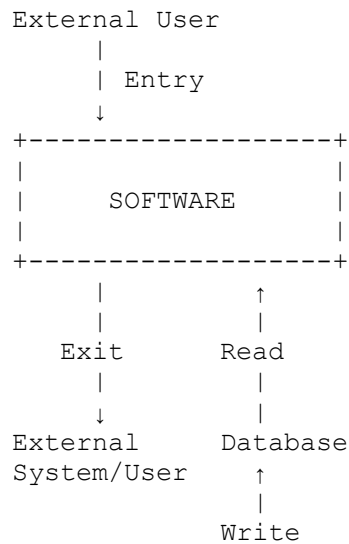
3. Data Movements

Movement of data:

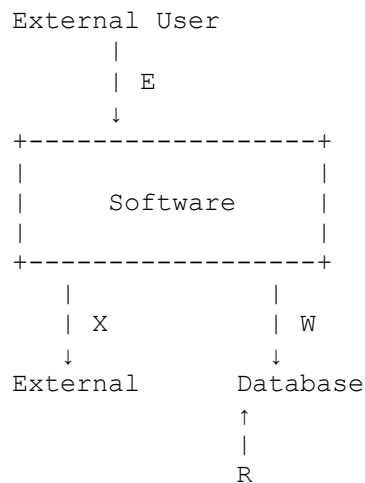
Entry → Exit → Read → Write

14. COSMIC Generic Software Model

A simple representation is:



More accurately:



15. COSMIC Measurement Process

COSMIC measurement generally involves four major activities.

Step 1: Define Measurement Purpose

Determine **why** you are measuring the software.

For example:

We want to estimate development effort and cost.

Step 2: Identify the Software Boundary

Determine what is inside and outside the software being measured.

Example:

Customer → [Online Shopping System] → Payment Gateway

The shopping system is the software being measured.

Step 3: Identify Functional Processes

Identify the major functional processes.

For an online shopping system:

- Login
 - Search Product
 - Add to Cart
 - Place Order
 - Make Payment
 - Track Order
-

Step 4: Identify Data Movements

For each functional process, identify:

- Entry
- Exit
- Read
- Write

Then count them.

16. Detailed Example: ATM Withdrawal

Consider an ATM withdrawal process.

Step 1

Customer enters card/PIN/withdrawal request.

Customer → ATM Software

Entry = 1

Step 2

ATM software reads customer/account information.

Database → ATM Software

Read = 1

Step 3

ATM software updates account balance.

ATM Software → Database

Write = 1

Step 4

ATM provides cash/transaction information to customer.

ATM Software → Customer

Exit = 1

Total

CFP=1E+1R+1W+1X 4 CFP

17. COSMIC vs Traditional Function Points

Students often confuse **COSMIC Function Points** with traditional **Function Point Analysis (FPA)**.

Traditional Function Points	COSMIC Function Points
Uses 5 function types	Uses 4 data movement types
EI, EO, EQ, ILF, EIF	Entry, Exit, Read, Write
Uses complexity weights	Each data movement is 1 CFP

Traditional Function Points

Often focuses on business applications
Based on IFPUG approach commonly

COSMIC Function Points

Suitable for many types of software
Based on COSMIC measurement method

18. COSMIC vs LOC

LOC

Measures lines of code

Language dependent

Difficult before coding

Measures implementation size

Same function can have different LOC

COSMIC

Measures functional size

More language independent

Can be estimated earlier

Measures functionality

Functionality can be measured consistently

Example

A login function could be implemented using:

- 100 lines of Java
- 50 lines of Python
- 150 lines of C++

LOC gives different measurements.

COSMIC focuses on the **functional data movements**, not the programming language.

19. Advantages of COSMIC

1. Language Independent

It does not depend on whether software is written in:

- Java
 - Python
 - C++
 - JavaScript
 - C#
-

2. Useful for Estimation

COSMIC size can be related to:

- Effort
- Cost
- Schedule
- Productivity

For example:

If historical productivity is:

10 CFP per person-month

and the project size is:

100 CFP

Then approximate effort:

100/10=10 person-months

3. Useful Early in Development

You can estimate functionality before knowing the exact implementation.

4. Suitable for Different Software Types

COSMIC can be applied to many types of software, including:

- Business systems
 - Real-time systems
 - Embedded systems
 - Web applications
 - Mobile applications
-

5. Helps Compare Productivity

Suppose:

Team A:

100 CFP / 10 person-months = 10 CFP/PM

Team B:

100 CFP / 20 person-months = 5 CFP/PM

Team A has higher measured productivity under the same measurement basis.

20. Limitations of COSMIC

1. Requires Training

Students and new project managers need to understand the measurement rules.

2. Identification Can Be Difficult

Correctly identifying functional processes and data movements requires analysis.

3. Does Not Directly Give Cost

COSMIC gives **functional size**.

You still need productivity/cost information to calculate effort and cost.

4. Historical Data Is Useful

To convert COSMIC size into effort, organizations benefit from historical productivity data.

21. COSMIC and Software Estimation

This is especially important in **Software Project Management**.

COSMIC itself primarily gives you:

Functional Size

Then historical productivity can be used to estimate effort.

Example

Suppose:

COSMIC size = 500 CFP

Historical productivity:

25 CFP/person-month

Then:

Effort=25500 20 person-months

If average cost is:

₹1 lakh/person-month

Then:

20×₹1,00,000 ₹20 lakh

Therefore:

COSMIC Size
↓
Productivity Data
↓
Effort
↓
Labour Cost
↓
Total Project Cost

22. Important Exam Example

Question:

Calculate the COSMIC functional size for a process having:

- 2 Entries
- 1 Exit
- 2 Reads
- 1 Write

Solution:

Formula:

$CFP = E + X + R + W$

Substitute:

$CFP = 2 + 1 + 2 + 1$ CFP=6

Therefore, the functional size is **6 COSMIC Function Points**.

23. Important Terminology

Term	Meaning
COSMIC	Common Software Measurement International Consortium
CFP	COSMIC Function Point
Functional Size	Amount of functionality provided by software
Entry (E)	Data enters software from external user/system
Exit (X)	Data leaves software to external user/system
Read (R)	Software retrieves data from persistent storage
Write (W)	Software stores data in persistent storage
Functional Process	Set of activities that accomplishes a meaningful task
Software Boundary	Boundary separating measured software from external entities

Dynamic Systems Development Method (DSDM)

1. Introduction

DSDM (Dynamic Systems Development Method) is an Agile software development method. It was developed to provide a more structured approach to rapid software development while still allowing flexibility and frequent user feedback.

It focuses on:

- Rapid delivery
- Active user involvement
- Iterative and incremental development
- Fixed time and cost
- High-quality software
- Continuous communication and collaboration

Simple Definition

DSDM is an Agile method that develops software in small increments through continuous user involvement, iterative development, timeboxing, and requirement prioritization.

2. Why Was DSDM Introduced?

Traditional development models such as Waterfall have a fixed sequence:

Requirements → Design → Coding → Testing → Deployment

The problem is that requirements may change during development. Making changes late can be expensive and time-consuming.

DSDM was introduced to solve this problem by:

- Developing the most important features first.
- Involving users throughout development.
- Delivering software in small increments.
- Accepting changes where possible.
- Keeping time and cost under control.

3. Main Idea of DSDM

The central idea of DSDM is:

Fix the time and cost, but prioritize and adjust the features.

For example, suppose a project must be completed in 6 months with a fixed budget.

Instead of extending the deadline when new requirements appear:

- Essential features → Must be completed
- Important features → Completed if possible
- Less important features → Can be postponed

This helps ensure that a working product is delivered on time.

4. Key Characteristics of DSDM

1. Active User Involvement

Users and customers participate throughout the project.

They:

- Provide requirements.
- Review prototypes.
- Give feedback.
- Help prioritize features.
- Test the developed system.

Example:

While developing a College Management System, college staff regularly review the attendance and fee modules and suggest improvements.

2. Iterative Development

The software is not developed only once from beginning to end.

Instead, the development cycle is repeated:

Build → Test → Get Feedback → Improve → Build Again

Each iteration improves the software.

3. Incremental Development

The complete system is divided into smaller parts called increments.

For example, in an Online Shopping Application:

Increment 1

- Registration
- Login

Increment 2

- Product Catalog
- Search

Increment 3

- Shopping Cart

Increment 4

- Payment

Each increment adds new functionality to the system.

4. Timeboxing

Timeboxing is one of the most important concepts in DSDM.

A fixed amount of time is allocated to complete a particular task or iteration.

Example:

Activity	Fixed Time
Requirement discussion	1 week
Prototype development	2 weeks
Testing	1 week
Deployment preparation	1 week

The time period is generally not extended easily. If some less important features cannot be completed, they are postponed.

Simple Meaning:

Time is fixed, but the amount of work may be adjusted according to priority.

5. Requirement Prioritization

DSDM uses the MoSCoW technique to prioritize requirements.

M — Must Have

Essential features without which the system cannot work.

Example:

A banking application must allow users to securely log in.

S — Should Have

Important features but the system can still operate temporarily without them.

Example:

Downloadable transaction statements.

C — Could Have

Desirable features that can be included if time and resources are available.

Example:
Dark mode.

W — Won't Have This Time

Features that are not included in the current release but may be developed later.

Example:
AI-based financial advice.

5. DSDM Principles

DSDM is based on eight main principles.

1. Focus on the Business Need

Every feature should provide value to the business or user.

The team should first develop the features that are most important.

Example:

For a food delivery application, ordering food is more important than adding multiple app themes.

2. Deliver on Time

The project should meet its agreed deadlines.

Timeboxes help the team control the schedule.

3. Collaborate

Developers, customers, managers, and other stakeholders should work closely together.

Good communication reduces misunderstandings.

4. Never Compromise Quality

Speed should not reduce software quality.

The team should define quality standards and maintain them throughout development.

5. Build Incrementally from Firm Foundations

The system should be developed step by step on a stable foundation.

Important architecture and business requirements should be understood before adding new features.

6. Develop Iteratively

The software is continuously improved through feedback.

Develop → Review → Improve → Repeat

7. Demonstrate Control

The project team should regularly monitor:

- Progress
- Cost
- Time
- Quality
- Risks

This helps ensure that the project remains under control.

8. Communicate Continuously and Clearly

Continuous communication between team members and stakeholders is essential.

Problems should be identified and solved quickly.

6. DSDM Project Lifecycle

The DSDM lifecycle consists of several phases.

Phase 1: Pre-Project

Purpose:

Ensure that only suitable projects are started.

Activities:

- Identify the business problem.
- Identify project objectives.
- Check whether DSDM is appropriate.
- Define initial project scope.

Example:

A company decides that it needs a Customer Support Portal.

Phase 2: Feasibility

Purpose:

Determine whether the project is feasible.

The team checks:

- Technical feasibility
- Financial feasibility
- Resource availability
- Time availability
- Major risks

Output:

A decision about whether to continue with the project.

Phase 3: Foundations

Purpose:

Create a strong foundation before detailed development begins.

Activities:

- Understand business requirements.
- Define system architecture.
- Identify major risks.
- Prioritize requirements using MoSCoW.
- Create an initial development plan.

Output:

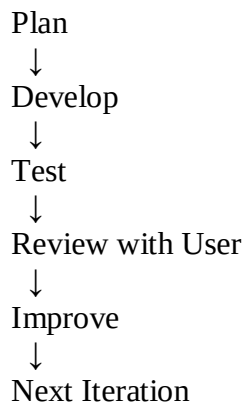
A clear foundation for development.

Phase 4: Evolutionary Development

This is the main development phase.

The software is developed through repeated iterations.

Cycle:



Each iteration produces a working part of the software.

Example:

Iteration 1: Login System
Iteration 2: User Dashboard
Iteration 3: Payment System
Iteration 4: Reports and Notifications

Phase 5: Deployment

The completed software is prepared for real users.

Activities:

- Final testing
- User Acceptance Testing (UAT)
- Data migration
- User training
- Deployment

The system is then released.

Phase 6: Post-Project

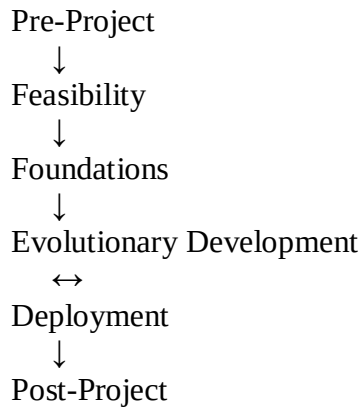
After deployment, the software is monitored and maintained.

Activities:

- Fix bugs
- Provide support
- Monitor performance

- Plan future releases
 - Add postponed features
-

7. DSDM Lifecycle Diagram



Evolutionary Development may involve several iterations and continuous feedback before deployment.

8. Roles in DSDM

DSDM involves different stakeholders.

Business Sponsor

- Provides funding.
 - Supports the project.
 - Ensures business benefits.
-

Business Visionary

- Defines the overall business vision.
 - Ensures that development supports business goals.
-

Business Ambassador

- Represents the users.
 - Provides regular feedback to the development team.
-

Project Manager

- Manages time, cost, resources, and risks.
 - Ensures that the project remains on track.
-

Technical Coordinator

- Provides technical guidance.
 - Ensures that the technical solution is appropriate.
-

Team Leader

- Coordinates the development team.
 - Helps organize day-to-day activities.
-

Developers

- Design and build the software.
-

Testers

- Test the software.
 - Ensure quality requirements are met.
-

9. Advantages of DSDM

1. Faster Delivery

Timeboxing helps deliver software within a fixed schedule.

2. Active User Participation

Continuous feedback increases customer satisfaction.

3. Flexible Requirements

Requirements can be changed and reprioritized.

4. Better Business Focus

Important features are developed first.

5. Controlled Cost and Time

The project focuses on fixed resources and deadlines.

6. Improved Quality

Testing and feedback occur throughout development.

7. Reduced Risk

Problems can be identified early.

10. Disadvantages of DSDM

1. Requires Active User Involvement

The method may fail if users are unavailable.

2. Requires Skilled Team Members

Team members need experience and strong communication skills.

3. Difficult for Very Large Projects

Coordination can become difficult.

4. Requirement Prioritization Can Be Difficult

Stakeholders may disagree about which features are most important.

5. Not Suitable for Every Project

Projects requiring completely fixed requirements and strict processes may need another model.

11. Example: Online Food Delivery Application

Suppose a company wants to develop a food delivery application in 4 months.

Step 1: MoSCoW Prioritization

Must Have

- User registration
- Restaurant listing
- Food ordering

- Payment

Should Have

- Order tracking
- Customer reviews

Could Have

- Dark mode
- Personalized recommendations

Won't Have This Time

- AI-based meal planning

Step 2: Timeboxing

Iteration	Duration	Features
Iteration 1	2 weeks	Registration and Login
Iteration 2	3 weeks	Restaurant and Menu
Iteration 3	3 weeks	Cart and Ordering
Iteration 4	2 weeks	Payment
Iteration 5	2 weeks	Tracking and Reviews

After every iteration, users provide feedback.

If time becomes limited, Could Have features are postponed, but Must Have features are completed.

This ensures a useful working application is delivered on time.

12. DSDM vs Scrum

DSDM	Scrum
Agile method	Agile framework
Strong focus on time and cost	Strong focus on Sprint delivery
Uses MoSCoW prioritization	Uses Product Backlog prioritization
Uses defined business and technical roles	Uses Product Owner, Scrum Master, and Developers
More structured governance	Simpler framework
Focuses strongly on business value	Focuses strongly on iterative product development

13. DSDM vs RAD

DSDM	RAD
Structured Agile method	Rapid development approach/process model
Uses timeboxing	Focuses mainly on rapid development
Uses MoSCoW prioritization	Often uses rapid prototyping
Strong project governance	Less structured
Focuses on business value and control	Focuses on speed and quick prototyping

Important Difference to Remember

RAD:

"Build software quickly."

DSDM:

"Build the most important software features quickly, within fixed time and cost."

Exam Answer Summary

Dynamic Systems Development Method (DSDM) is an Agile software development method that emphasizes rapid and iterative development, active user involvement, timeboxing, incremental delivery, and prioritization of requirements using the MoSCoW technique.

Key Features

- Timeboxing
- MoSCoW prioritization
- Active user involvement
- Iterative development
- Incremental delivery
- Continuous testing
- Strong business focus

DSDM Phases

Pre-Project → Feasibility → Foundations → Evolutionary Development → Deployment → Post-Project

One-Line Memory Formula

DSDM = Agile + User Involvement + Timeboxing + MoSCoW + Iterative Development + Fast Delivery

Effort and Cost Estimation Techniques in Software Project Management

1. Introduction

In **Software Project Management (SPM)**, one of the most important tasks of a project manager is to estimate:

- **How much work is required?** → Effort estimation
- **How much time is required?** → Schedule estimation
- **How much money is required?** → Cost estimation

The basic relationship can be remembered as:

Software Size → Effort → Schedule → Cost

2. What is Effort Estimation?

Effort estimation is the process of predicting the amount of human work required to complete a software project.

Effort is generally measured in:

- Person-hours
- Person-days
- Person-months

Example

Suppose a project requires:

- 4 developers
- 5 months

Then approximate effort is:

$4 \times 5 = 20$ person-months

So, the estimated effort is **20 person-months**.

Important: Person-months measure effort, not necessarily calendar time.

3. What is Cost Estimation?

Cost estimation is the process of predicting the total financial resources required to complete a software project.

It may include:

- Employee salaries
- Hardware
- Software licenses
- Cloud infrastructure
- Testing
- Training
- Travel
- Communication
- Maintenance
- Contingency/reserve

Basic Formula

Total Cost=Labour Cost+Other Project Costs

4. Relationship Between Effort and Cost

Effort is one of the major inputs for calculating cost.

For example:

Estimated effort:

20 person-months

Average labour cost:

₹80,000 per person-month

Then:

$$20 \times ₹80,000 = ₹16,00,000$$

So labour cost = **₹16 lakh.**

If other costs are ₹4 lakh:

$$₹16L + ₹4L = ₹20L$$

Therefore:

Total estimated project cost = ₹20 lakh

5. Major Effort and Cost Estimation Techniques

The important techniques are:

1. **Expert Judgment**
2. **Estimation by Analogy**
3. **Top-Down Estimation**
4. **Bottom-Up Estimation**
5. **Work Breakdown Structure (WBS)**
6. **Parametric/Algorithmic Models**
7. **COCOMO**
8. **Function Point-Based Estimation**
9. **Three-Point / PERT Estimation**
10. **Price-to-Win Estimation**

Let's understand each one.

6. Expert Judgment

Definition

In **Expert Judgment**, experienced professionals estimate the effort and cost based on their knowledge and previous experience.

Experts may include:

- Project managers
- Senior developers
- Software architects
- Technical leads

Example

A project manager has previously developed three similar e-commerce applications.

Based on that experience, they estimate:

Effort = 15 person-months

Duration = 5 months

Cost = ₹12 lakh

Advantages

- Simple.
- Fast.
- Useful when historical data is limited.
- Uses practical experience.

Disadvantages

- Subjective.
 - Different experts may give different estimates.
 - Highly dependent on expert experience.
 - Personal bias can affect estimation.
-

7. Estimation by Analogy

Definition

In this technique, the current project is compared with a **previously completed similar project**.

Example

Previous project:

College Management System

- Size: 25 KLOC
- Effort: 18 person-months
- Cost: ₹14 lakh

New project:

University Management System

It is slightly larger and more complex.

The manager may estimate:

- Effort $\approx$ 25 person-months
- Cost $\approx$ ₹20 lakh

Advantages

- Easy to understand.
- Uses real project data.
- Often more reliable than pure guessing.

Disadvantages

- Requires similar historical projects.
 - Projects may not actually be comparable.
 - Differences in technology and team skills can affect accuracy.
-

8. Top-Down Estimation

Definition

In **Top-Down estimation**, the project is estimated **as a whole first**, and then the total effort/cost is distributed among different components.

Example

Estimated total project effort:

100 person-months

The manager distributes it:

Module	Estimated Effort
User Management	15 PM
Product Management	25 PM
Payment	20 PM
Reports	15 PM
Testing	15 PM
Deployment	10 PM
Total	100 PM

Advantages

- Fast.
- Useful during early project planning.
- Good when detailed requirements are unavailable.

Disadvantages

- Individual tasks may be underestimated.
 - Less accurate for complex projects.
 - Depends heavily on overall project experience.
-

9. Bottom-Up Estimation

Definition

In **Bottom-Up estimation**, the project is divided into small tasks. Each task is estimated separately, and the estimates are added together.

Example

For an online shopping system:

Task	Effort
Login	2 PM
Product Catalog	4 PM
Shopping Cart	3 PM
Payment	5 PM
Testing	4 PM
Deployment	2 PM
Total	20 PM

Therefore:

Total estimated effort = 20 person-months

Advantages

- More detailed.
- Often more accurate.
- Helps identify individual resource requirements.
- Useful for scheduling.

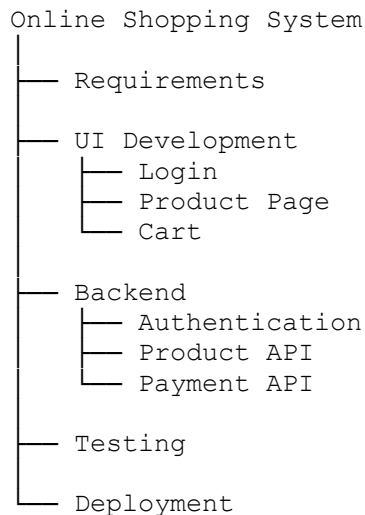
Disadvantages

- Time-consuming.
 - Requires detailed requirements.
 - Some small tasks may be forgotten.
-

10. Work Breakdown Structure (WBS)

WBS is a technique for breaking a large project into smaller manageable components.

Example



Each work package is estimated separately.

Then:

Total Effort = $\sum$ Effort of all tasks

WBS is particularly useful with **bottom-up estimation**.

11. Parametric / Algorithmic Estimation

Definition

An **algorithmic model** uses mathematical equations based on measurable project characteristics.

General idea:

Effort = $f(\text{Software Size, Complexity, Other Factors})$

One of the most famous algorithmic models is **COCOMO**.

12. COCOMO Model

COCOMO stands for:

Constructive Cost Model

It was developed by **Barry Boehm**.

COCOMO estimates:

- Effort
- Development time
- Cost

based mainly on software size, usually measured in **KLOC**.

Basic COCOMO Formula

$$E = a(KLOC)^b$$

Where:

- **E** = Effort in person-months
- **KLOC** = Thousand Lines of Code
- **a, b** = constants depending on project type

Development time can be estimated using:

$$T = c(E)^d$$

Where:

- **T** = Development time in months
- **E** = Effort
- **c, d** = model constants

Then staffing can be approximated as:

$$\text{Average Staff} = TE$$

13. COCOMO Project Types

Basic COCOMO classifies projects into three categories.

1. Organic

Suitable for:

- Small projects
- Experienced teams
- Familiar technology
- Relatively simple requirements

Example:

Small payroll system.

2. Semi-Detached

Suitable for projects with:

- Medium complexity
- Mixed team experience
- Moderate size

Example:

Medium-scale business application.

3. Embedded

Suitable for:

- Highly complex systems
- Strict hardware/software constraints
- High reliability requirements

Example:

Aircraft control system.

14. Function Point Estimation

Definition

Function Point Analysis (FPA) estimates software size based on the **functionality delivered to users**, rather than lines of code.

It considers five major components:

1. **External Inputs (EI)**
2. **External Outputs (EO)**
3. **External Inquiries (EQ)**
4. **Internal Logical Files (ILF)**
5. **External Interface Files (EIF)**

Example: Banking System

Inputs:

- Deposit amount
- Withdrawal request

Outputs:

- Account statement
- Transaction report

Inquiries:

- Check balance

Internal files:

- Customer database

External interfaces:

- External payment system

These are assigned weights based on complexity to calculate function points.

15. Advantages of Function Point Estimation

- Independent of programming language.
- Can be used early in the project.
- Measures user functionality.
- Useful when LOC cannot be estimated reliably.

Disadvantages

- Requires experience.
 - Counting can be subjective.
 - More difficult for beginners.
-

16. Three-Point Estimation

Software estimates contain uncertainty.

Instead of making only one estimate, three estimates are made:

Optimistic (O)

Best-case scenario.

Most Likely (M)

Most realistic estimate.

Pessimistic (P)

Worst-case scenario.

A commonly used PERT estimate is:

$$E=6O+4M+P$$

Example

Suppose development of a module is estimated as:

- O = 5 days
- M = 8 days
- P = 14 days

Then:

$$E=6(5)+4(8)+14 \quad E=65 \quad E=8.5 \text{ days}$$

So expected effort/duration is approximately **8.5 days**.

17. Price-to-Win Estimation

This is slightly different from estimating the actual development cost.

Here, the project price is determined based on **what the customer or market is willing to pay**.

Example

The actual estimated development cost is:

₹15 lakh

But the customer says:

"Our maximum budget is ₹12 lakh."

The company may decide whether it can deliver within ₹12 lakh.

Risk

The company may win the contract but make very little profit or even a loss.

18. Cost Estimation Components

When calculating total project cost, consider:

1. Labour Cost

Developer and staff salaries.

2. Hardware Cost

Servers, computers, networking equipment.

3. Software Cost

Licenses, development tools, databases, etc.

4. Cloud Cost

Hosting, storage, databases, APIs, etc.

5. Training Cost

Training employees on new technology.

6. Testing Cost

Testing tools, testers, environments.

7. Communication Cost

Internet, meetings, collaboration tools.

8. Contingency

Additional reserve for unexpected problems.

19. Example of Complete Cost Estimation

Suppose:

Development Team

5 developers $\times$ ₹70,000/month $\times$ 6 months

=₹21,00,000

Testing

=₹3,00,000

Cloud and Hardware

=₹2,00,000

Software Licenses

=₹1,00,000

Contingency

=₹3,00,000

Therefore:

Total Cost=21+3+2+1+3 =₹30 lakh

20. Effort vs Cost Estimation

Effort Estimation	Cost Estimation
Measures human work	Measures money required
Person-hours/days/months	Rupees/dollars

Effort Estimation	Cost Estimation
Focuses on labour	Includes labour + other expenses
Example: 30 person-months	Example: ₹25 lakh
Used for staffing and planning	Used for budgeting and pricing

21. Comparison of Techniques

Technique	Main Idea	Advantage	Limitation
Expert Judgment	Expert experience	Quick	Subjective
Analogy	Compare previous projects	Practical	Needs similar data
Top-Down	Estimate whole project	Fast	Less detailed
Bottom-Up	Estimate individual tasks	Detailed	Time-consuming
WBS	Break project into work packages	Organized	Requires detailed planning
COCOMO	Mathematical model	Systematic	Depends on reliable size estimate
Function Points	Estimate functionality	Language-independent	Requires expertise
Three-Point	O, M, P estimates	Handles uncertainty	More calculation
Price-to-Win	Based on customer budget	Competitive	May reduce profit

22. Important Factors Affecting Estimation

Even a good technique can produce an inaccurate estimate if important factors are ignored.

Technical Factors

- Complexity
- Technology
- Integration requirements
- Performance requirements

Human Factors

- Team experience
- Developer productivity
- Team size
- Communication

Project Factors

- Requirement stability
- Schedule constraints
- Quality requirements
- Project size

External Factors

- Market conditions
 - Vendor dependencies
 - Regulations
 - Customer availability
-

23. Why Software Estimation Is Difficult

Software estimation is difficult because:

Requirements may change

Customer requirements can evolve.

Software is intangible

Unlike construction, it is difficult to physically measure software progress.

Productivity differs

Two developers may take different amounts of time for the same task.

Technology may be new

New frameworks or tools create uncertainty.

Hidden work exists

Integration, debugging, documentation, meetings, deployment, and rework may be underestimated.

Therefore:

Software estimation should be treated as an evolving prediction, not an exact number.

24. Best Practice: Use More Than One Technique

In real projects, managers often combine techniques.

For example:

WBS + Expert Judgment + Analogy + Three-Point Estimation

This gives a more reliable estimate than depending on a single method.

Example

For a payment module:

- Expert estimate = 10 days
- Historical project = 12 days
- Three-point estimate = 11 days

The manager reviews all three and produces a final estimate based on project context.

25. Easy Flow to Remember

```
Understand Requirements
↓
Estimate Software Size
↓
Select Estimation Technique
↓
Estimate Effort
↓
Estimate Resources
↓
Estimate Schedule
↓
Calculate Labour Cost
↓
Add Other Costs
↓
Add Risk/Contingency
↓
Final Project Cost
```

Extreme Programming (XP)

1. Introduction

Extreme Programming (XP) is an **Agile software development method** that focuses on producing **high-quality software through continuous feedback, close teamwork, frequent testing, and simple design.**

XP is called “**Extreme**” because it takes good software development practices and applies them **very frequently or intensively.**

Simple Definition

Extreme Programming (XP) is an **Agile method that improves software quality and responsiveness to changing requirements through practices such as pair programming, continuous testing, frequent releases, and continuous integration.**

2. Main Objective of XP

The main objectives of XP are:

- Develop high-quality software.
 - Handle changing requirements easily.
 - Reduce software defects.
 - Improve communication between developers and customers.
 - Deliver working software frequently.
 - Keep the design simple.
-

3. How XP Works

XP follows a continuous cycle:

Requirements → Planning → Design → Coding → Testing → Customer Feedback → Next Release

The process is repeated several times.

User Requirements



Planning



Simple Design



Coding



Testing



Customer Feedback



Small Release



Repeat / Improve

4. Core Values of XP

XP is based on **five core values**:

1. Communication

Developers, testers, managers, and customers communicate continuously.

Example:

Instead of waiting until the end, the developer can directly discuss a changing requirement with the customer.

2. Simplicity

Develop only what is required at the present time.

Do not add unnecessary features.

Simple design = Easier development + Easier maintenance

3. Feedback

Continuous feedback is collected from:

- Customers
- Test results
- Other developers
- The running software

This helps identify problems early.

4. Courage

Developers should have the courage to:

- Change old code when required.
 - Accept changing requirements.
 - Remove unnecessary code.
 - Report problems honestly.
-

5. Respect

All team members should respect each other's ideas and contributions.

XP depends heavily on teamwork.

5. Important XP Practices

1. Pair Programming

In **Pair Programming**, two developers work together on one computer.

Roles:

Driver

- Writes the code.

Navigator

- Reviews the code.
- Thinks about design and possible errors.

After some time, they can switch roles.

Benefits:

- Fewer coding errors.

- Better knowledge sharing.
 - Continuous code review.
 - Improved code quality.
-

2. Test-Driven Development (TDD)

In TDD, developers **write the test before writing the actual code**.

Process:

Write Test → Test Fails → Write Code → Test Passes → Refactor

Example:

Suppose we need to create a function to add two numbers.

Step 1: Write a test:

add(2, 3) should return 5.

Step 2: The test fails because the function does not exist.

Step 3: Write the required code.

Step 4: Run the test again.

Step 5: Improve the code if necessary.

Advantage:

Defects are detected early.

3. Continuous Integration

Developers integrate their code into a shared system frequently.

After integration:

- The software is built.
- Automated tests are executed.
- Errors are detected early.

Example:

Instead of waiting one month to combine all modules, developers may integrate their work **daily or multiple times during development**.

4. Small Releases

XP delivers software in **small and frequent releases**.

Example:

For an Online Shopping Website:

Release 1: Registration and Login

Release 2: Product Listing

Release 3: Shopping Cart

Release 4: Payment System

The customer can start using important features early.

5. Simple Design

XP follows the principle:

Design the simplest solution that works.

Developers should avoid unnecessary complexity.

Example:

If a basic login system meets the current requirement, there is no need to initially build a highly complex authentication system with unnecessary features.

6. Refactoring

Refactoring means improving the internal structure of existing code **without changing its external functionality**.

Example:

Before refactoring:

Long and complex code

After refactoring:

Small functions

Better naming

Reusable code

Improved structure

The software should still produce the same output.

Benefits:

- Cleaner code
 - Easier maintenance
 - Better readability
 - Reduced complexity
-

7. Collective Code Ownership

In XP, the code belongs to the **whole development team**.

Any authorized team member can improve any part of the code when needed.

This prevents dependency on only one developer.

8. Coding Standards

All developers follow common coding rules.

For example:

- Same naming conventions.
- Same code formatting.
- Proper comments where needed.
- Common project structure.

This makes the code easier for all team members to understand.

9. On-Site Customer

A customer or customer representative should be closely available to the development team.

The customer helps:

- Clarify requirements.
- Answer questions.
- Prioritize features.
- Provide continuous feedback.

10. Sustainable Pace

The team should maintain a reasonable and sustainable working speed.

Continuous excessive overtime can reduce:

- Productivity
 - Code quality
 - Developer performance
-

6. XP Development Process

Step 1: User Stories

Requirements are written as simple **User Stories**.

Example:

As a customer, I want to search for products so that I can find items quickly.

Step 2: Release Planning

The team selects which user stories will be included in the next release.

Step 3: Iteration Planning

The selected work is divided into smaller development tasks.

Step 4: Design and Coding

Developers create a simple design and start development.

XP practices such as:

- Pair Programming
- TDD
- Coding Standards

are followed.

Step 5: Continuous Testing

Tests are performed continuously.

If a test fails, the problem is fixed immediately.

Step 6: Customer Feedback

The working software is shown to the customer.

Feedback is collected.

Step 7: Small Release

A working version of the software is released.

Then the cycle starts again.

7. XP Practices Summary Table

XP Practice	Meaning
Pair Programming	Two developers work together
TDD	Write tests before code
Continuous Integration	Frequently combine and test code
Small Releases	Deliver software frequently
Simple Design	Build the simplest solution
Refactoring	Improve code without changing functionality
Collective Ownership	Team can improve shared code
Coding Standards	Follow common coding rules
On-Site Customer	Customer provides continuous feedback
Sustainable Pace	Maintain a reasonable work speed

8. Advantages of XP

1. High Software Quality

Continuous testing and pair programming help reduce errors.

2. Easy to Handle Changes

Customer feedback allows requirements to change during development.

3. Early Detection of Bugs

TDD and continuous testing identify problems quickly.

4. Frequent Software Delivery

Small releases provide working software early.

5. Better Team Collaboration

Pair programming and collective ownership improve teamwork.

6. Better Code Quality

Refactoring and coding standards keep the code clean and maintainable.

9. Disadvantages of XP

1. Requires Active Customer Involvement

XP works best when customers are available regularly.

2. Pair Programming Can Be Expensive

Two developers working together may increase development cost.

3. Requires Skilled Developers

Developers need discipline and experience.

4. Less Documentation

XP focuses more on working software, which may result in limited documentation.

5. Difficult for Large Distributed Teams

Continuous communication can become difficult when teams are very large or geographically distributed.

10. Example: Online Food Delivery Application

Suppose a team is developing a **Food Delivery App** using XP.

User Story:

As a customer, I want to order food online.

Pair Programming:

Two developers work together to develop the order feature.

TDD:

They first write tests for placing an order.

Simple Design:

They create only the necessary order functionality.

Continuous Integration:

The order module is frequently integrated with the main application.

Customer Feedback:

The customer requests an additional **Order Cancellation** feature.

Small Release:

The basic ordering system is released first.

The new feature is added in the next iteration.

XP vs Scrum**Extreme Programming (XP)**

Focuses strongly on engineering practices

Uses TDD and Pair Programming

Focuses on code quality

Defines coding practices

Frequent technical feedback

Scrum

Focuses strongly on project management

Uses Sprints and Scrum events

Focuses on managing and delivering work

Does not prescribe specific coding techniques

Frequent sprint-based feedback

Easy to Remember:

Scrum tells the team how to manage the work.

XP tells developers how to develop high-quality software.

Software Estimation in Software Project Management

Software estimation is one of the most basic and important activities in **Software Project Management (SPM)**. Before starting a project, the manager needs to estimate **how much work is required, how long it will take, how many people are needed, and how much it will cost**.

Simple Definition

Software estimation is the process of predicting the effort, time, resources, and cost required to develop a software project.

1. Why Do We Need Software Estimation?

Imagine a client says:

"Build me an online shopping website."

The project manager cannot simply start coding. They need to answer:

- How much software needs to be developed?
- How many developers are required?
- How long will development take?
- How much will it cost?
- What resources are required?
- What risks can affect the estimate?

Therefore, estimation is done **before and during the project**.

2. What Do We Estimate?

There are four major things students should remember:

Size → Effort → Time → Cost

```
Software Size
  ↓
  Effort
  ↓
Development Time
  ↓
  Cost
```

These are closely related.

A. Software Size

Size represents how large the software is.

Common measures include:

- **LOC** – Lines of Code
- **KLOC** – Thousand Lines of Code
- **Function Points (FP)**
- **Use Case Points (UCP)**

Example

If estimated software size is:

20 KLOC

it means approximately **20,000 lines of source code**.

Important: LOC is usually an estimate, not something known accurately before development.

3. Effort Estimation

Effort means the amount of human work required to complete the project.

It is commonly measured in:

- Person-hours
- Person-days
- Person-months

Example

Suppose:

5 developers × 4 months = 20 person-months

So estimated effort = **20 person-months**.

Important:

Effort ≠ Duration

If a project requires 20 person-months, it does **not automatically mean** that 20 people can finish it in one month.

Some tasks depend on each other, and adding people can also increase communication overhead.

4. Time / Schedule Estimation

This determines **how long the project will take**.

Example:

Estimated duration = 6 months

The project manager creates a schedule containing:

- Requirements
 - Design
 - Development
 - Testing
 - Deployment
-

5. Cost Estimation

Cost estimation determines the **total money required** for the project.

Costs can include:

- Developer salaries
- Testing
- Hardware
- Cloud services
- Software licenses
- Office/infrastructure
- Training
- Maintenance
- Contingency

Simple Example

Suppose:

Cost	Amount
------	--------

Cost	Amount
Development	₹10 lakh
Testing	₹2 lakh
Cloud/Hardware	₹1 lakh
Other expenses	₹1 lakh
Total	₹14 lakh

Estimated project cost = **₹14 lakh**.

6. Main Software Estimation Techniques

There are several ways to estimate software projects.

1. Expert Judgment

An experienced developer/project manager estimates the project based on previous experience.

Example:

"I have developed similar systems before. This project will probably require around 8 person-months."

Advantage:

Simple and quick.

Disadvantage:

Depends heavily on the expert's experience and can be subjective.

2. Estimation by Analogy

The current project is compared with a **similar completed project**.

Example:

Previous project:

College Management System → 10 person-months

New project is similar but slightly larger.

Estimated effort might be around:

12–14 person-months.

Advantage:

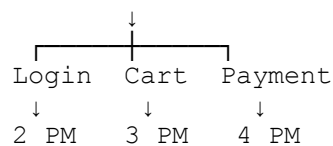
Useful when historical project data is available.

3. Decomposition Technique

The project is divided into smaller components, and each component is estimated separately.

Example:

Online Shopping System



Total:

2 + 3 + 4 = 9 person-months

This is called **bottom-up estimation**.

4. LOC-Based Estimation

Estimate the number of lines of code and then calculate effort.

For example:

Estimated size:

20 KLOC

If historical data tells us that a particular team requires a certain amount of effort for each KLOC, that information can be used to estimate total effort.

Problem:

It is difficult to accurately predict LOC before development.

5. Function Point (FP) Estimation

Function Point measures software size based on the **functionality provided to the user**, rather than lines of code.

It considers things such as:

- Inputs
- Outputs
- User inquiries
- Internal files
- External interfaces

Example

For a banking application:

- Login
- Deposit
- Withdrawal
- Balance inquiry
- Transaction statement

Each contributes to the functional size of the system.

Advantage:

It can be used before the programming language is selected.

6. COCOMO Model

COCOMO = Constructive Cost Model

It was developed by **Barry Boehm**.

COCOMO estimates:

- Effort
- Development time
- Project cost

based mainly on estimated software size.

The basic COCOMO effort equation is:

$$E=a(KLOC)^b$$

Where:

- **E** = Effort in person-months
- **KLOC** = Thousand Lines of Code
- **a, b** = model constants

For basic COCOMO, projects are commonly categorized as:

- Organic
 - Semi-detached
 - Embedded
-

7. Three-Point Estimation

Instead of making one estimate, three estimates are considered:

Optimistic (O)

Best-case estimate.

Most Likely (M)

Most realistic estimate.

Pessimistic (P)

Worst-case estimate.

A commonly used PERT-style expected estimate is:

$$E=6O+4M+P$$

Example

Suppose a task has:

- O = 4 days
- M = 6 days
- P = 10 days

Then:

$$E=64+4(6)+10 \quad E=638 \approx 6.33 \text{ days}$$

This technique considers **uncertainty and risk**.

8. What Affects Software Estimation?

Estimation is affected by many factors.

1. Project Size

Larger projects generally require more effort.

2. Complexity

A simple website is easier to estimate than a banking or AI system.

3. Team Experience

Experienced developers may complete tasks faster.

4. Technology

New technologies may introduce uncertainty.

5. Requirements

Changing or unclear requirements make estimation difficult.

6. Quality Requirements

Higher security, performance, and reliability requirements increase effort.

7. Tools and Infrastructure

Availability of development tools and infrastructure affects productivity.

8. Risk

Technical, organizational, and external risks can increase effort and cost.

9. Estimation vs Planning

Students often confuse these two.

Estimation

Answers:

"How much will the project require?"

Examples:

- 20 person-months
- 6 months
- ₹15 lakh

Planning

Answers:

"How will we complete the project?"

Planning includes:

- Assigning people
- Creating schedule
- Assigning tasks
- Setting milestones
- Managing resources

Simple Difference

Estimation = Predict

Planning = Organize

10. Estimation vs Scheduling

Estimation

Project requires **20 person-months** of effort.

Scheduling

Development will run from January to May, with 5 developers working on different tasks.

So:

Estimation tells us the expected effort/time/cost.

Scheduling decides when activities will happen and who will perform them.

11. Estimation Process

A simple software estimation process is:

```
Understand Requirements
      ↓
Estimate Software Size
      ↓
Estimate Effort
      ↓
Estimate Schedule
      ↓
Estimate Cost
      ↓
Review Risks & Assumptions
      ↓
Finalize Estimate
```

12. Important Point: Estimation Is Not Exact

Software estimation is a **prediction**, not a guarantee.

For example, saying:

"The project will definitely take exactly 5 months."

is usually unrealistic.

A better approach is:

"Based on current requirements and assumptions, the estimated duration is approximately 5 months."

As the project progresses and more information becomes available, estimates can be **revised**.

13. Example

Suppose a company wants to develop a **College Management System**.

The manager estimates:

- Software size = 30 KLOC
- Effort = 25 person-months
- Duration = 6 months
- Team = 5 people
- Development cost = ₹12 lakh
- Testing cost = ₹2 lakh
- Infrastructure = ₹1 lakh

Total Estimated Cost

$$\text{₹12L} + \text{₹2L} + \text{₹1L} = \text{₹15 lakh}$$

The manager can now prepare the project plan and budget.

Managing Iterative Processes in Software Project Management

1. What is an Iterative Process?

An **iterative process** is a software development approach in which the software is developed through **repeated cycles (iterations)**.

Instead of trying to develop the complete software in one cycle, the team:

Plans → Develops → Tests → Reviews → Gets Feedback → Improves → Repeats

Each iteration produces an improved version of the software.

Simple Example

Suppose we are developing an **Online Shopping System**.

Instead of developing everything together:

- **Iteration 1:** Login & Registration
- **Iteration 2:** Product Search & Product Details
- **Iteration 3:** Shopping Cart
- **Iteration 4:** Payment
- **Iteration 5:** Order Tracking

Each iteration adds functionality to the previous version.

2. What Does "Managing Iterative Processes" Mean?

In **Software Project Management (SPM)**, managing an iterative process means **planning, organizing, monitoring, controlling, and reviewing each iteration** so that the project progresses toward its final objectives.

The project manager must ensure:

- The right work is selected for each iteration.
- The iteration finishes within the planned time.
- Resources are available.
- Risks are controlled.
- Quality is maintained.
- Customer feedback is incorporated.

- The project remains within budget and schedule.

3. Why Use an Iterative Process?

Traditional development may follow:

Requirements → Design → Coding → Testing → Final Delivery

The customer may not see the actual working system until very late.

An iterative approach provides:

```
Iteration 1 → Feedback
      ↓
Iteration 2 → Feedback
      ↓
Iteration 3 → Feedback
      ↓
Iteration 4 → Final Product
```

Major Benefits

- Early delivery of working software.
- Early identification of defects.
- Easier handling of changing requirements.
- Continuous customer involvement.
- Better risk management.
- Better estimation as the project progresses.

4. Iterative Process Management Cycle

A project manager generally manages iterations through the following activities:

```
Iteration Planning
      ↓
Task Allocation
      ↓
Development
      ↓
Testing
      ↓
Monitoring
      ↓
Review / Demonstration
      ↓
Customer Feedback
      ↓
Retrospective
```

↓
Plan Next Iteration

This cycle continues until the required product is completed.

5. Step 1 — Iteration Planning

The first step is to decide **what should be developed during the iteration**.

The project manager and team consider:

- Product requirements
- Customer priorities
- Available resources
- Previous iteration results
- Technical dependencies
- Risks
- Estimated effort

Example

Suppose the product backlog contains:

Requirement	Priority
Login	High
Product Search	High
Cart	High
Dark Mode	Low
AI Recommendations	Low

For the next iteration, the team may select:

Login + Product Search

because they have higher priority.

6. Step 2 — Define Iteration Goals

Each iteration should have a **clear and measurable goal**.

Poor Goal:

Improve the application.

Good Goal:

Implement and test user registration and login functionality.

A clear goal helps the project manager determine whether the iteration was successful.

7. Step 3 — Break Work into Tasks

Large requirements are divided into smaller tasks.

Example: Login Feature

```
Login Feature
↓
UI Design
↓
Database Design
↓
Backend API
↓
Authentication
↓
Frontend Integration
↓
Testing
```

Breaking work into smaller tasks makes it easier to:

- Estimate effort.
 - Assign responsibilities.
 - Track progress.
 - Identify delays.
-

8. Step 4 — Estimate Effort

The team estimates how much effort each task requires.

Common estimation techniques include:

- Story points
- Expert judgment
- Planning poker
- Function points
- Historical data

Example

Task	Estimated Effort
Login UI	4 hours
Login API	6 hours
Database	3 hours
Testing	4 hours

Total estimated effort = **17 hours**

The project manager compares this with the team's available capacity.

9. Step 5 — Resource Allocation

The project manager assigns tasks according to:

- Skills
- Availability
- Experience
- Workload

Example

Team Member	Task
Developer 1	Frontend
Developer 2	Backend
Database Developer	Database
Tester	Testing

The manager must avoid overloading one team member while others have little work.

10. Step 6 — Development

The team starts implementing the selected features.

During development:

- Developers write code.
- Designers create interfaces.
- Database developers create required structures.
- Testers prepare test cases.

The project manager monitors progress without unnecessarily interfering with technical work.

11. Step 7 — Continuous Testing

Testing should occur **throughout the iteration**, not only at the end of the entire project.

Testing may include:

- Unit testing
- Integration testing
- Functional testing
- Regression testing
- User acceptance testing

Why?

If a defect is discovered early, it is generally easier to fix than discovering it several iterations later.

12. Step 8 — Monitor Iteration Progress

The project manager continuously checks:

Schedule

Is the team on time?

Cost

Are we staying within the budget?

Scope

Are we developing only the agreed features?

Quality

Are defects increasing?

Resources

Are team members available?

Risks

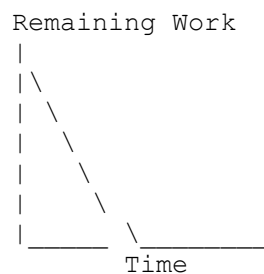
Have new risks appeared?

13. Progress Tracking

Various tools can be used to monitor progress.

Burndown Chart

A **burndown chart** shows how much work remains during an iteration.



Ideally, remaining work should decrease as the iteration progresses.

14. Step 9 — Managing Changes

One of the biggest advantages of iterative development is the ability to handle changing requirements.

However, **every change should not automatically be accepted.**

The project manager should evaluate:

- Business value
- Cost
- Time impact
- Technical impact
- Risk
- Dependencies

Example

Customer requests:

"Add online payment during the current iteration."

The manager evaluates whether:

- The team has capacity.
- Payment affects existing modules.
- Security requirements are satisfied.
- The change can be completed without affecting the iteration goal.

If not, it may be moved to the next iteration.

15. Step 10 — Iteration Review

At the end of an iteration, the team demonstrates the completed functionality to stakeholders.

Review Questions

- What was completed?
- What was not completed?
- Does the feature meet requirements?
- What does the customer think?
- What should be changed?

Example

The team demonstrates the Login module.

Customer says:

"Login works, but we also need OTP-based authentication."

The requirement can then be added or prioritized for a future iteration.

16. Step 11 — Customer Feedback

Customer feedback is one of the most important parts of iterative development.

Feedback may result in:

- New requirements.
- Requirement changes.
- UI improvements.
- Bug fixes.
- Priority changes.

The updated requirements become input for future iterations.

17. Step 12 — Iteration Retrospective

A **retrospective** is a meeting where the team evaluates its own working process.

Three common questions are:

What went well?

Example:

Testing was completed early.

What did not go well?

Example:

Backend development was delayed.

What should we improve?

Example:

Start API development earlier in the next iteration.

The goal is **continuous process improvement**.

18. Managing Risks in Iterative Processes

Iterative development helps identify risks early.

Example

Suppose a project requires a new payment technology.

Instead of waiting until the final stage, the team can test the payment technology in an early iteration.

If it fails:

- The problem is discovered early.
- Alternative technology can be investigated.
- Project risk is reduced.

Common Risks

- Technical risks
 - Requirement changes
 - Resource shortage
 - Schedule delays
 - Integration problems
 - Security problems
 - Performance problems
-

19. Managing Scope

Scope management is extremely important.

Without proper control, continuous changes can cause **scope creep**.

Example

Original requirement:

Build an online shopping website.

Customer gradually asks for:

- Chat system
- AI recommendation
- Voice search
- Loyalty points
- Live tracking
- Social media integration

If everything is immediately added, the project may never finish.

Therefore, the project manager prioritizes requirements and decides **what belongs in the current iteration and what should be postponed**.

20. Managing Quality

Quality must be maintained in every iteration.

The project manager should ensure:

- Coding standards are followed.
- Testing is performed.

- Defects are tracked.
- Code reviews are performed.
- Acceptance criteria are satisfied.

Important Principle

Do not postpone quality until the final iteration.

21. Managing Communication

Iterative projects require frequent communication between:

- Project manager
- Developers
- Testers
- Customers
- Business stakeholders

Communication may happen through:

- Daily meetings
- Sprint/iteration reviews
- Progress reports
- Issue trackers
- Project management tools
- Team discussions

Good communication helps identify problems early.

22. Managing Dependencies

Some tasks cannot start until another task is completed.

Example

```
Database Design
  ↓
Backend API
  ↓
Frontend Integration
  ↓
Testing
```

The project manager must identify such dependencies and plan accordingly.

Otherwise, a delay in one task can delay the entire iteration.

23. Managing Multiple Iterations

The project manager must also manage the **overall project**, not just individual iterations.

Example:

Iteration	Goal	Status
1	Login	Completed
2	Product Catalog	Completed
3	Cart	In Progress
4	Payment	Planned
5	Order Tracking	Planned

The manager continuously compares actual progress with the overall project plan.

24. Definition of Done

Before declaring an iteration complete, the team should define what "**Done**" means.

For example, a feature is considered complete only when:

- Coding is completed.
- Code review is completed.
- Unit tests pass.
- Integration testing is completed.
- Major defects are fixed.
- Customer acceptance criteria are satisfied.
- Documentation is updated where required.

This prevents the team from claiming incomplete work as finished.

25. Example: College Management System

Suppose a college wants to develop a **College Management System**.

Iteration 1

Goal: Student Registration

Develop:

- Registration form
- Database
- Validation
- Testing

Customer reviews it.

Iteration 2

Goal: Attendance

Develop:

- Attendance entry
- Attendance calculation
- Reports

Customer provides feedback.

Iteration 3

Goal: Examination

Develop:

- Marks entry
 - Result calculation
 - Result report
-

Iteration 4

Goal: Fee Management

Develop:

- Fee records
- Payment
- Receipts

Each iteration produces a working improvement to the system.

26. Advantages of Managing Iterative Processes

1. Early Delivery

Working functionality becomes available early.

2. Early Risk Identification

Problems are discovered sooner.

3. Better Requirement Management

Changing requirements can be incorporated.

4. Continuous Customer Feedback

The product is continuously reviewed.

5. Better Quality

Testing happens repeatedly.

6. Improved Estimation

Future iterations can be estimated using actual project performance.

7. Better Visibility

Stakeholders can see actual progress rather than relying only on reports.

27. Challenges

Managing iterative processes also has challenges:

- Frequent requirement changes.
- Difficulty maintaining long-term schedules.
- Scope creep.
- Continuous customer involvement required.
- Team coordination.

- Repeated testing and integration.
- Managing technical debt.
- Maintaining architecture while requirements evolve.

28. Traditional vs Iterative Project Management

Traditional	Iterative
Plan most things upfront	Plan progressively
Large final delivery	Frequent smaller deliveries
Changes are difficult	Changes are expected
Feedback often comes late	Feedback comes frequently
Testing may occur later	Continuous testing
Risk may be discovered late	Risks identified earlier
Progress based heavily on planned tasks	Progress demonstrated through working software

29. Important Terms to Remember

Term	Meaning
Iteration	A development cycle that produces an improved product
Iteration Goal	What the team intends to achieve
Backlog	Prioritized list of work
Timebox	Fixed period for completing work
Increment	Working addition to the product
Review	Evaluation of completed work
Retrospective	Evaluation of the development process
Feedback	Input used to improve future iterations
Definition of Done	Conditions required to consider work complete
Scope Creep	Uncontrolled expansion of project requirements